

Artificial Intelligence Laboratory — Logical Planning

Using an LLM to Construct and Test a Simple Planning Agent

Warehouse Robot Package Delivery

1. Task 0 — Understand the Planning Problem

(a) Initial state I

{At(Robot, A), At(Package, A)}

(b) Goal G

{At(Package, C)}

(c) Available actions

- Move(A, B)
- Move(B, A)
- Move(B, C)
- Move(C, B)
- Pickup(Package, A)
- Drop(Package, C)

(d) Preconditions and effects

Action	Preconditions	Positive effects	Negative effects
Move(A,B)	At(Robot,A)	At(Robot,B)	At(Robot,A)
Move(B,A)	At(Robot,B)	At(Robot,A)	At(Robot,B)
Move(B,C)	At(Robot,B)	At(Robot,C)	At(Robot,B)
Move(C,B)	At(Robot,C)	At(Robot,B)	At(Robot,C)
PickUp(Package,A)	At(Robot,A), At(Package,A)	Holding(Package)	At(Package,A)
Drop(Package,C)	At(Robot,C), Holding(Package)	At(Package,C)	Holding(Package)

Initial applicability

PickUp(Package,A) is applicable because both of its positive preconditions, At(Robot,A) and At(Package,A), are present in the initial state.

Drop(Package,C) is not applicable initially because its preconditions require both At(Robot,C) and Holding(Package), neither of which is true initially.

2. Task 1 — Construct a Plan by Hand

A valid plan is to pick up the package at A, move the robot through B to C, and then drop the package at C:

PickUp(Package,A) → Move(A,B) → Move(B,C) → Drop(Package,C)

State	Action just applied	Facts
S0	—	At(Robot,A), At(Package,A)
S1	PickUp(Package,A)	At(Robot,A), Holding(Package)
S2	Move(A,B)	At(Robot,B), Holding(Package)
S3	Move(B,C)	At(Robot,C), Holding(Package)
S4	Drop(Package,C)	At(Robot,C), At(Package,C)

3. Task 2 — LLM Prompt and Generated Planner

The exact specification prompt used for the LLM was:

I want to implement a simple planning agent in Python for this warehouse domain.

Initial state:

{At(Robot,A), At(Package,A)}

Goal:

{At(Package,C)}

Locations are A, B, and C. The robot can move only along:

A <-> B and B <-> C.

Actions:

- Move(A,B): precondition At(Robot,A); effect remove At(Robot,A), add At(Robot,B)*
- Move(B,A): precondition At(Robot,B); effect remove At(Robot,B), add At(Robot,A)*
- Move(B,C): precondition At(Robot,B); effect remove At(Robot,B), add At(Robot,C)*
- Move(C,B): precondition At(Robot,C); effect remove At(Robot,C), add At(Robot,B)*
- PickUp(Package,A): preconditions At(Robot,A) and At(Package,A); effects remove At(Package,A), add Holding(Package)*
- Drop(Package,C): preconditions At(Robot,C) and Holding(Package); effects remove Holding(Package), add At(Package,C)*

Represent a state as a set of logical propositions.

Represent each action with:

- a name*
- positive preconditions*
- negative preconditions*
- positive effects*
- negative effects*

An action is applicable only if all positive preconditions are present and all negative preconditions are absent.

When an action is applied:

- 1. remove its negative effects;*

2. add its positive effects.

Use breadth-first search to find a sequence of actions that reaches the goal.

The program must:

- detect when no plan exists;*
- print the action sequence;*
- print the state after every action;*
- make it possible to verify every returned action independently.*

Also explain the implementation and identify assumptions.

Implementation mapping

Specification concept	Implementation
State	A frozenset of logical proposition strings.
Positive/negative preconditions	Stored separately in each Action object.
Effects	Positive effects are added; negative effects are removed.
Applicability	Action.applicable() checks all preconditions.
Goal	The planner checks whether the goal facts are a subset of the current state.
Search	Breadth-first search uses a queue and records parent states.
No-plan detection	BFS returns None when the queue is exhausted.
Plan verification	verify_plan() re-executes each action from the initial state.

The implementation and notebook were generated/structured with LLM assistance, then executed and independently checked using explicit assertions and separate plan verification.

4. Task 3 — Test the Generated Planner

Test	Initial state	Goal	Plan found?	Valid?
A — Solvable	{At(Robot,A), At(Package,A)}	{At(Package,C)}	Yes	Yes
B — Impossible	{At(Robot,A), At(Package,A)}	{At(Package,C)}	No	N/A — no plan
C — Irrelevant action	{At(Robot,A), At(Package,A)}	{At(Package,C)}	Yes	Yes

Test A — Solvable problem

Result: PASS

Plan: Pickup(Package,A) → Move(A,B) → Move(B,C) → Drop(Package,C)

The independently executed verifier confirmed every action was applicable and that the final state satisfied At(Package,C).

Test B — Impossible problem

The PickUp action was removed. The robot can still move, but it can never obtain Holding(Package), so Drop(Package,C) can never become applicable. The planner correctly returned “No plan found” instead of inventing an action.

Test C — Irrelevant actions

An additional movement action was introduced that moves the robot but does not move the package. The planner still required the actual package goal At(Package,C), so reaching C with the robot alone was not treated as success. The returned plan was independently verified.

5. Task 4 — Logic and Search

The logical component determines whether an action can legally be applied in the current state. For a state S and action a , this is the check $S \models \text{Preconditions}(a)$. If the check succeeds, the action's effects produce a successor state.

The search component decides which sequence of applicable actions to explore. BFS maintains a queue of previously discovered states and systematically explores alternatives until a state satisfying the goal is found.

Completed planning flow:

*Current state → Check action preconditions → Applicable? → Generate successor state
→ Search over alternatives → Goal?*

In other words, logic determines what is possible; search determines what to try. The two components work together because search is restricted to transitions that are logically valid.

6. Task 5 — Can the LLM Verify Its Own Plan?

An LLM can explain why a proposed plan appears valid by listing each action's preconditions and describing the state before the action. However, an explanation generated by the same system is not an independent verification.

The independently executed Python state transitions should be trusted more for verification because they mechanically apply the specified preconditions and effects to the actual current state. This follows the laboratory's central engineering principle: a generated explanation is not the same as independent verification.

7. Reflection Questions

1. Why is it useful to specify action preconditions and effects before asking an LLM to write the planner?

Preconditions and effects define the legal transitions of the planning problem. Giving them explicitly constrains the LLM's implementation and provides a precise specification against which the generated program can be checked. It also prevents the planner from treating an action as applicable merely because its name appears in the action list.

2. Give an example of an error that could occur if the planner failed to check an action's preconditions.

The planner could apply Drop(Package,C) while the robot is still at A and is not holding the package. That would incorrectly place the package at C without the required physical sequence of actions.

3. Why is a plan that “looks reasonable” not necessarily a valid plan?

A plan must satisfy every action's preconditions at the exact state where the action is executed and its effects must lead to the next state. A sequence can sound plausible while containing an inapplicable action or an incorrect state transition.

4. What did the LLM contribute to the implementation?

The LLM translated the formal planning specification into Python structures for actions and states, implemented applicability and state-transition logic, and produced the BFS planning framework and output routines.

5. What did you have to verify independently?

The action definitions, applicability checks, state transitions, returned plan, final goal satisfaction, failure case, and irrelevant-action behaviour were independently executed and tested. The final plan was also re-executed with a separate verification function.

6. In this laboratory, where is logical reasoning being used?

Logical reasoning is used when the planner checks whether all action preconditions are satisfied and when it applies positive and negative effects to derive the next state.

7. How is planning related to the search algorithms studied in the previous module?

Planning can be formulated as search over a state space. Each state is a set of logical facts, each applicable action generates a successor state, and BFS searches sequences of such actions until it reaches a state satisfying the goal.